

FIT BUDDY AI

8-Phase Project Documentation

AI Fitness Plan Generator using Gemini Models

Project Category: Generative AI

Team Leader: Mariya Darshini (25ds022)

Team Members: S. Harini (25ds016), R. Subhashri (25ds041), S. Mythili (25ds024)

Technology: FastAPI, Google Gemini, SQLAlchemy, SQLite, Jinja2

GitHub: <https://github.com/Mariyadarshini/fitbuddy-ai>

How the project follows the 8 phases

Phase	Title	Main Evidence
1	Brainstorming & Idea Generation	Idea, problem, proposed solution, selection rationale
2	Requirement Analysis	Problem statement, functional/non-functional, software/hardware/user requirements
3	Project Design	Architecture, modules, UI, data design, flowchart and algorithm
4	Project Planning	WBS, timeline, task allocation, milestones and risk log
5	Project Development	Actual source code, folders, Gemini integration and working UI
6	Project Testing	test_app.py, test cases, validation, fixes and outputs
7	Project Documentation	README, report, diagrams, screenshots and references
8	Project Demonstration	Working website flow, plan outputs and demo-video link

PHASE 1 – Brainstorming & Idea Generation

1.1 Project Idea

FitBuddy AI is a Generative AI-powered fitness planning web application. It helps users create a personalized 7-day workout plan and a nutrition or recovery tip based on their age, weight, fitness goal and preferred workout intensity, and lets them refine the plan through feedback.

1.2 Problem Identified

- Users may not know which workouts suit their age, weight, goal and fitness level.
- Generic workout plans do not adapt when the user's needs or preferences change.
- Workout planning, nutrition guidance and recovery advice are usually scattered across different sources.
- Coaches or admins need one place to view users and their plans.

1.3 Proposed Solution

FitBuddy AI collects the user's details and uses Google Gemini to generate a structured 7-day plan with warm-up, main workout, cooldown and recovery tip for each day. It also provides a short nutrition or recovery tip, stores everything in SQLite, and allows feedback-based plan updates.

1.4 Why FitBuddy AI Was Selected

- It fits the Generative AI project category.
- It combines AI integration with frontend, backend and database development.
- It solves a practical, everyday problem.
- It contains several modules that can be demonstrated separately.
- It provides a foundation for future features such as BMI calculation and progress tracking.

1.5 Brainstorming Notes

The team considered an AI workout plan generator, a nutrition and recovery tip assistant, feedback-based plan revision, an admin or coach dashboard, BMI and basic fitness calculation, and progress tracking with workout history. The first four were selected for the working application; BMI calculation and progress tracking were kept for future scope.

PHASE 2 – Requirement Analysis

2.1 Problem Statement

To develop an AI-assisted web application that accepts a user's details, fitness goal and workout intensity and generates a personalized 7-day workout plan and nutrition or recovery tip, with support for feedback-based plan updates and stored user data.

2.2 Functional Requirements

- User enters name, user ID, age, weight, fitness goal and workout intensity.
- System generates a 7-day plan with warm-up, workout, cooldown and recovery tip for each day.
- System provides a nutrition or recovery tip based on the goal.
- User can submit feedback and receive an updated plan.
- User profiles and plans are stored in a SQLite database.
- Admin or coach can view all users and their plans.
- JSON API endpoints and Swagger documentation are available.
- The application works with a fallback generator when Gemini is unavailable.

2.3 Non-Functional Requirements

- Simple and responsive interface.
- Input validation for all fields.
- Reliable operation through the fallback generator.
- Maintainable, modular Python code.
- Secure handling of the API key through environment variables.
- General wellness guidance only – not medical advice.

2.4 Software Requirements

Component	Requirement
Language	Python 3.11
Backend framework	FastAPI with Uvicorn
Templates	Jinja2, HTML, CSS
Database	SQLite with SQLAlchemy
Validation	Pydantic and pydantic-settings
AI SDK	google-genai (Google Gemini)
Testing	Pytest and HTTPX / FastAPI TestClient
Browser	Any modern web browser

2.5 Hardware Requirements

- A computer able to run Python 3.11 (the project was run on Windows).
- Internet connection when the Gemini API is used; the fallback mode works offline.

2.6 User Requirements

- Users need a browser and their basic details: name, user ID, age, weight, goal and intensity.
- Admins or coaches need access to the Users page to review plans.
- A Gemini API key is optional and is set in the local .env file.

PHASE 3 – Project Design

3.1 System Architecture

The browser sends requests to the FastAPI backend. Routes call the Gemini AI service, which falls back to the local generator if needed. Generated plans are validated, stored in SQLite through SQLAlchemy and displayed through Jinja2 templates.

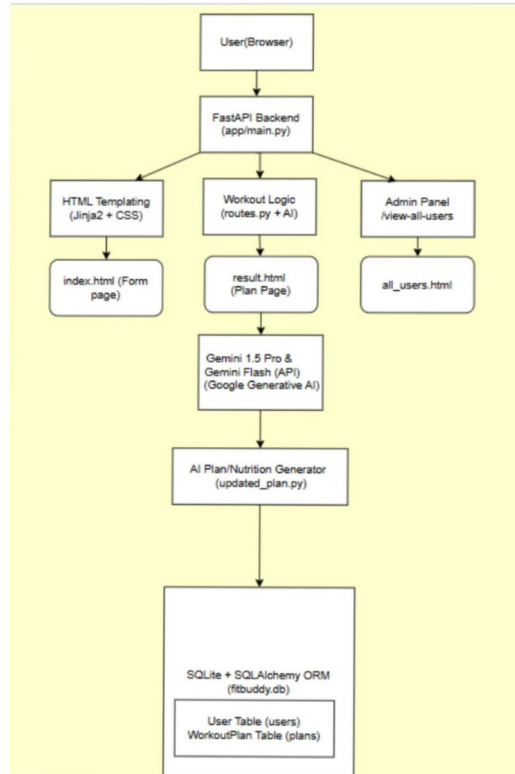


Figure 3.1 – FitBuddy AI architecture diagram

3.2 Module Design

Module	Responsibility
app/main.py	FastAPI app, static files and database initialization at startup
app/routes.py	Web form routes and JSON API routes
app/schemas.py	Pydantic models for input, plan structure and responses
app/models.py	SQLAlchemy models User and WorkoutPlan
app/crud.py	Database create, read and update operations
app/services/gemini_service.py	Gemini calls for workout, nutrition tip and plan revision
app/services/fallback_generator.py	Local plan and tip generator used when Gemini is unavailable
app/services/serializers.py	Conversion between plan objects and stored JSON

Method	Endpoint	Purpose
GET	/	Home page with planner form
POST	/generate-workout	Generate plan from the web form
POST	/submit-feedback	Update plan from the feedback form
GET	/view-all-users	Admin view of users and plans
GET	/api/health	Health check

Method	Endpoint	Purpose
POST	/api/generate-plan	Generate plan (JSON)
GET	/api/nutrition-tip	Nutrition / recovery tip for a goal
POST	/api/update-plan/{user_id}	Update plan with feedback (JSON)

3.3 UI Design

The interface has a Home page with the planner form, a Result page showing the 7-day plan, nutrition tip and feedback box, a Users page for the admin view and an Error page. The header provides Generate, Users and API Docs links, and the footer states that the content is general wellness guidance, not medical advice.

3.4 Data Design

Table	Fields
users	id, name, age, weight, goal, intensity, created_at, updated_at
workout_plans	id, user_id (foreign key to users), original_plan, updated_plan, nutrition_tip, generation_mode, created_at, updated_at

One user can have many workout plans. Plans are stored as JSON text.

3.5 Flowchart

The complete flowchart is provided as Flowchart.svg in the Project Design and Project Documentation folders.

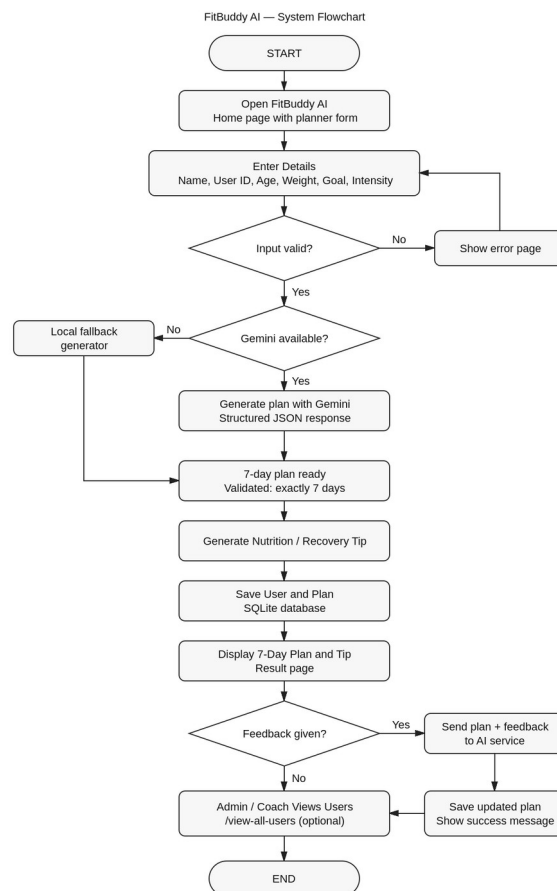


Figure 3.2 – FitBuddy AI system flowchart

3.6 Algorithm

1. Start the application; the database tables are created.
2. Display the Home page and collect user details.
3. Validate the input; show the error page if invalid.
4. Generate the 7-day plan with Gemini, or with the fallback generator if Gemini is unavailable.
5. Validate that the plan contains exactly 7 days.
6. Generate the nutrition or recovery tip.
7. Store the user and plan in SQLite.
8. Display the plan and tip.
9. If feedback is submitted, revise the plan and store the updated plan.
10. Display the updated plan with a success message.
11. Allow the admin or coach to view all users on the Users page.

PHASE 4 – Project Planning

4.1 Work Breakdown Structure (WBS)

WBS No.	Work Package	Activities
1	Project initiation	Idea selection, problem definition
2	Requirements	Functional and non-functional requirements
3	Design	Architecture, modules, database, UI, flowchart
4	Development	Backend, services, Gemini integration, templates
5	Testing	Pytest test cases, validation, fixes
6	Documentation	Report, README, screenshots
7	Demonstration	Working application, demo recording

4.2 Suggested Project Timeline

Period	Planned Work
Week 1	Brainstorming and topic selection
Week 2	Requirement analysis and initial design
Week 3	Project structure, database models and schemas
Week 4	Plan generation, nutrition tip and fallback generator
Week 5	Gemini integration, feedback update and templates
Week 6	Testing, debugging and UI refinement
Week 7	Documentation, screenshots and report
Week 8	Final demonstration and submission

4.3 Task Allocation

Team Member	Role / Responsibility
Mariya Darshini (25ds022)	Team Leader – overall coordination
S. Harini (25ds016)	Team member – development, testing and documentation support
R. Subhashri (25ds041)	Team member – development, testing and documentation support
S. Mythili (25ds024)	Team member – development, testing and documentation support

4.4 Milestones

- Idea and requirements finalized.
- Database, schemas and routes completed.
- Gemini integration with fallback completed.
- Templates and admin view completed.
- Pytest tests passed.
- Report and presentation prepared.
- Application demonstrated.

4.5 Risk Log

Risk	Level	Mitigation
Gemini API key missing or quota exceeded	High	Automatic fallback generator keeps the application working.
AI model names change	Medium	Model IDs are configurable in the .env file.
Invalid user input	Medium	Pydantic validation and error page.
AI returns an invalid plan	Medium	Response is validated against a schema requiring exactly 7 days; fallback is used on failure.
Health-related misuse	Medium	Prompts avoid medical claims and the interface shows a wellness disclaimer.

PHASE 5 – Project Development

5.1 Actual Project Structure

```
fitbuddy-ai/
├── app/
│   ├── __init__.py, main.py, config.py, database.py
│   ├── models.py, schemas.py, crud.py, routes.py
│   └── services/
│       ├── gemini_service.py
│       ├── fallback_generator.py
│       └── serializers.py
├── templates/ (base, index, result, all_users, error .html)
├── static/css/style.css
├── tests/ (conftest.py, test_app.py)
├── data/
├── .env.example, .gitignore, requirements.txt, pyproject.toml
├── start.ps1, start.bat
└── README.md
```

5.2 Backend Development

The backend is built with FastAPI. `routes.py` contains the web form routes and the JSON API routes. Pydantic schemas validate input (age 13–100, weight 20–400 kg, intensity low, medium or high) and the plan structure. SQLAlchemy models `User` and `WorkoutPlan` store data in SQLite, and `crud.py` handles saving, updating and reading users and plans.

5.3 Gemini AI Integration

`gemini_service.py` uses the google-genai SDK. Three functions call the model: `generate_workout` creates the 7-day plan as structured JSON, `generate_nutrition_tip` returns one short tip, and `update_workout` revises the plan using the user's feedback. The workout and nutrition model IDs are read from the `.env` file, so they can be changed without editing code. If the API key is missing or the call fails, the local fallback generator provides the plan and tip.

5.4 Frontend Development

The frontend uses Jinja2 templates with a shared `base.html` and a single `style.css`. `index.html` holds the planner form, `result.html` shows the plan, nutrition tip and feedback box, `all_users.html` is the admin view, and `error.html` shows validation or processing errors.

PHASE 6 – Project Testing

6.1 Testing File

Testing is done with Pytest in tests/test_app.py, using FastAPI TestClient. conftest.py sets an empty Gemini key, enables fallback mode and uses a separate test database, so the tests run without a Gemini API key.

6.2 Test Case Table

ID	Test	Expected Result	Status
TC01	Open Home page (GET /)	Status 200 and page contains FitBuddy	Pass
TC02	Health check (GET /api/health)	Status 200 and status is ok	Pass
TC03	Generate plan (POST /api/generate-plan)	Status 200, 7 days and a nutrition tip returned	Pass
TC04	Update plan (POST /api/update-plan/1001)	Status 200 and updated plan has 7 days	Pass
TC05	Nutrition tip (GET /api/nutrition-tip)	Status 200 and tip returned	Pass

6.3 Validation

- Age must be between 13 and 100.
- Weight must be greater than 20 and at most 400 kg.
- Intensity must be low, medium or high.
- Name and goal must not be empty.
- Feedback must have at least 3 characters.
- The generated plan must contain exactly 7 days.

6.4 Issues Found and Fixes

- The original project notes used the discontinued google-generativeai package and older model names. The implementation uses the current google-genai SDK with configurable model IDs.
- To avoid failures when the API key is missing or quota is exhausted, an automatic fallback generator was added.

6.5 Final Testing Evidence

The pytest run completed with all 5 tests passing (100%).

```
• (.venv) PS C:\Users\acer\Downloads\fitbuddy-ai\fitbuddy-ai> python -c "from app.database import init_db; init_db(); print('Database initialized successfully')"
```

```
Database initialized successfully
```

```
• (.venv) PS C:\Users\acer\Downloads\fitbuddy-ai\fitbuddy-ai> pytest -q
```

```
..... [100%]
```

```
===== warnings summary =====
```

Figure 6.1 – Database initialization and pytest run

PHASE 7 – Project Documentation

7.1 Documentation Included

- Aim.txt – project aim.
- Algorithm.txt – step-by-step algorithm.
- Flowchart.svg – project flowchart.
- Results.txt – observed results.
- Project_Report.docx – full project report.
- FitBuddy_AI_8_Phase_Project_Documentation (this document, Word and PDF).
- README.md files in the project root and in the development folder.

7.2 Report Contents

The project report covers the project overview and description, four usage scenarios, main features, technical architecture, workout plan generation, nutrition tip, feedback-based revision, terminal and interface screenshots, and the conclusion.

7.3 Documentation Evidence

- GitHub repository: <https://github.com/Mariyadarshini/fitbuddy-ai>
- Architecture diagram and interface screenshots are included in the Design and Demonstration folders.
- The .env.example file is provided; the real .env file must not be uploaded.

PHASE 8 – Project Demonstration

8.1 Demonstration Flow

1. Start the server with `uvicorn app.main:app --reload`.
2. Open the FitBuddy AI home page in the browser.
3. Enter name, user ID, age, weight, goal and intensity.
4. Click Generate 7-Day Plan and review the plan.
5. Show the nutrition or recovery tip.
6. Enter feedback and click Update My Plan.
7. Show the success message and updated plan.
8. Open the Users page and the Swagger docs (/docs).

8.2 Final Output Evidence

The screenshot displays the FitBuddy AI home page. At the top, there's a dark green header with the logo 'FitBuddy AI' and navigation links for 'Generate', 'Users', and 'API Docs'. The main heading is 'Your personalized 7-day workout plan.' with a subtext: 'Enter your details, choose an intensity, and FitBuddy builds a structured weekly plan plus a nutrition or recovery tip.' To the right, a dark green box highlights '7 days' of 'Personalized guidance'. The central form, titled 'Create your plan', includes input fields for 'Name' (Arjun), 'User ID' (10), 'Age' (25), and 'Weight (kg)' (70). It also has a 'Fitness goal' text area with the placeholder 'e.g. muscle gain, weight loss, flexibility, general fitness' and a 'Workout intensity' dropdown menu set to 'Medium'. A green button labeled 'Generate 7-Day Plan --' is at the bottom of the form. Below the form, three light green boxes provide additional information: 'AI plan' (Day-by-day warm-up, workout, cooldown and recovery guidance), 'Feedback loop' (Tell FitBuddy what to change and regenerate the plan), and 'Persistent' (User profiles and generated plans are stored in SQLite). A footer note states: 'FitBuddy AI - General wellness guidance, not medical advice.'

Figure 8.1 – Home page with planner form

FitBuddy AI Generate Users API Docs

PERSONALIZED PLAN

Arun's 7-day fitness plan New plan

Your plan has been updated based on your feedback!

Generation mode: **fallback** · Model: **local-fallback**

User information

Name Arun	User ID 10	Age 25
Weight 70.0 kg	Goal muscle gain	Intensity Medium

Workout plan

A 7-day medium-intensity plan for FitBuddy user, age 30, 70.0 kg, with the goal: general fitness. Build consistency first and adjust the workload to your experience. Feedback incorporated locally: add more cardio and make day 4 lighter

Day 1 Full body

Warm-up	Workout	Cooldown
• 5–8 min easy walking	• Full body: 3 rounds at a sustainable pace	• 3–5 min easy walking

Figure 8.2 – Generated 7-day fitness plan

Nutrition / recovery tip

Aim to eat a palm-sized portion of protein—like chicken, eggs, tofu, or beans—with every main meal to give your muscles the building blocks they need to grow. Pair this with quality sleep so your body can properly repair and recover between workouts.

Figure 8.3 – Nutrition / recovery tip

Share your feedback

Ask for changes such as more cardio, less volume, an extra rest day, or a stronger mobility focus.

e.g. Add more cardio and make Day 4 lighter...

Update My Plan →

Figure 8.4 – Feedback form

Your plan has been updated based on your feedback!

Figure 8.5 – Plan update confirmation

8.3 Demo Video Link

The Google Drive demo video link is provided in Project Demo Link.txt in the Project Demonstration folder.

8.4 Final Demonstration Statement

FitBuddy AI demonstrates how Generative AI can be integrated into a FastAPI web application to provide personalized fitness plans. The project shows a complete workflow: collecting user details, generating a 7-day plan, giving a nutrition or recovery tip, storing data in SQLite and updating the plan from user feedback.

APPENDIX – Evidence Checklist

Evidence Item	Location	Status
Source code	5. Project Development Phase / fitbuddy-ai	Included
Architecture diagram	3. Project Design Phase / Solution Architecture.pdf	Included
Flowchart	Flowchart.svg (Design and Documentation folders)	Included
UI screenshots	3. UI_Design and 8. Demo Screenshots	Included
Test file and pytest run	6.Project Testing	Included
Project report	7. Project Documentation / Project_Report.docx	Included
GitHub link	Project Demo Link.txt	Included
Demo video link	Project Demo Link.txt	Add link